Extending Kubernetes StatefulSets

CLOUD
KITCHENS

Who are we?

This is us

Augusto Ribeiro
Software Engineer
Storage

Jakob Schultz-Falk
Software Engineering Manager
Storage

CLOUD KITCHENS™

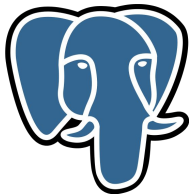
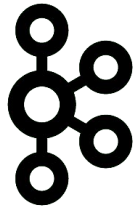
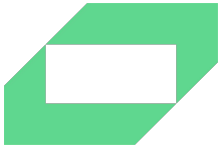
- Delivery only kitchens
- Order Aggregation
- Point of sales systems
- Back office analytics

Who are we?

Infra Core Storage @ CloudKitchens

- Storage on Kubernetes
- CockroachDB, PostgreSQL, Vespa, Clickhouse, ValKey, Kafka, Blob Storage, KV
- We are not DBAs (at least not good ones)
- We would rather automate than operate
- So we build Kubernetes operators



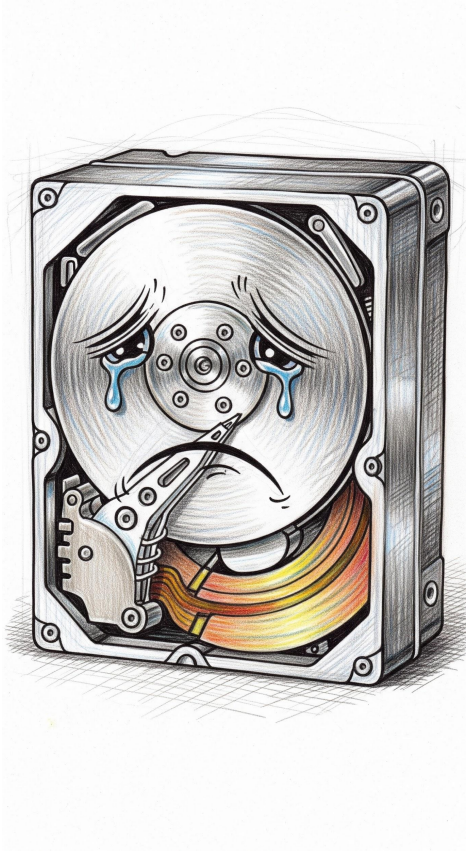





The Problem

What are we fixing?

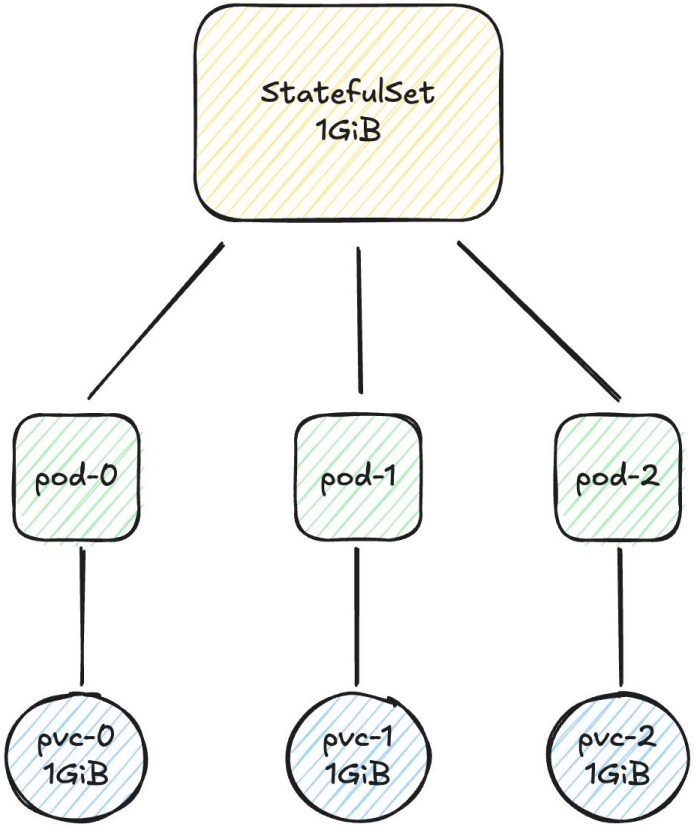
Kubernetes was not built for storage

- StatefulSets have immutable volume templates so it doesn't even support expansion (thought this is being worked on in enhancement 4650)
- We can expand a single PersistentVolumeClaim
- But explicit non-goals for disk shrinking
- Since there is no declarative approach every change is manual
- Each change is tedious and yet stressful
- Resulting in a high-risk of incidents with data-loss



So you want more disk, huh?

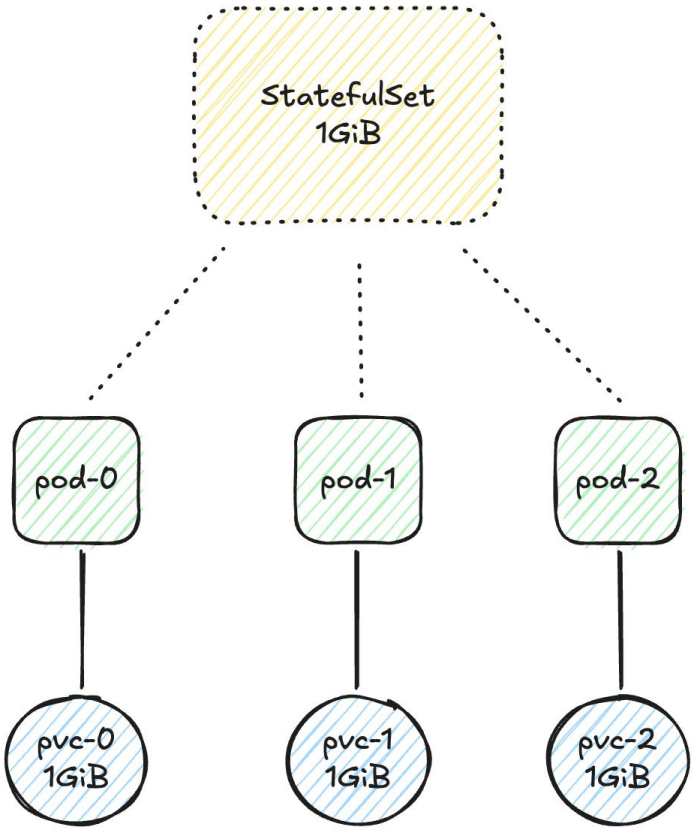
The manual approach to expanding disks on a StatefulSet



So you want more disk, huh?

The manual approach to expanding disks on a StatefulSet

1. Delete the StatefulSet
    - **N.B.** Remember to orphan the pods so your service doesn't go away!

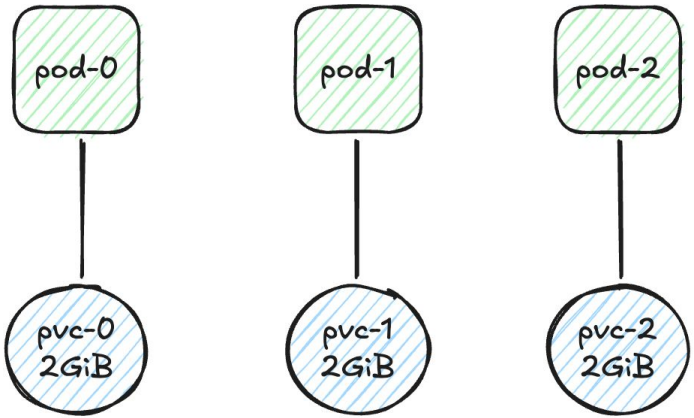


So you want more disk, huh?

The manual approach to expanding disks on a StatefulSet

1. Delete the StatefulSet
   - **N.B.** Remember to orphan the pods so your service doesn't go away!

2. Update each individual PVC with the new requested capacity and wait for the CSI driver to expand the disk



So you want more disk, huh?

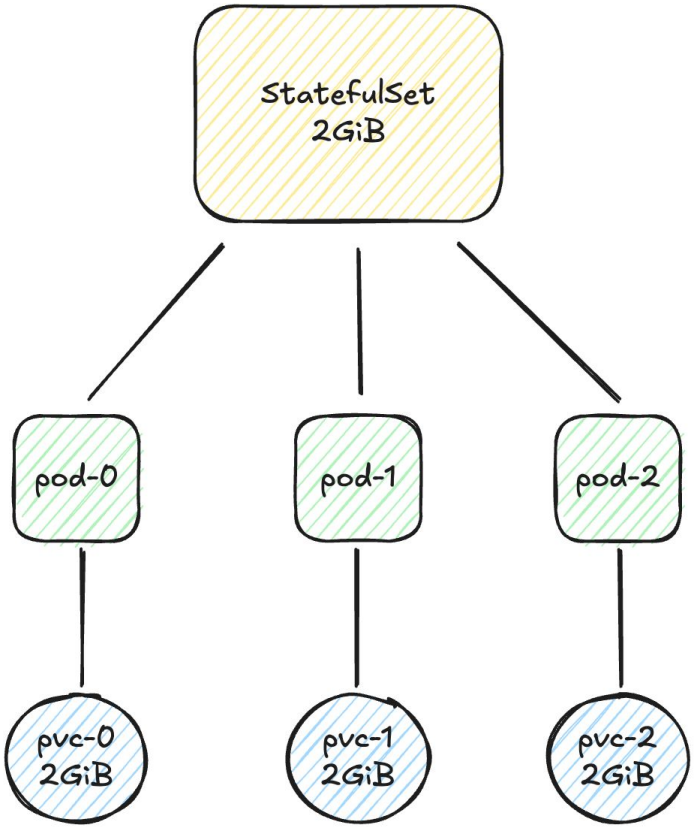
The manual approach to expanding disks on a StatefulSet

1. Delete the StatefulSet
   - **N.B.** Remember to orphan the pods so your service doesn't go away!

2. Update each individual PVC with the new requested capacity and wait for the CSI driver to expand the disk

3. Recreate the StatefulSet with the updated size in the volume claim template

4. Repeat ad-nauseum



The Objectives

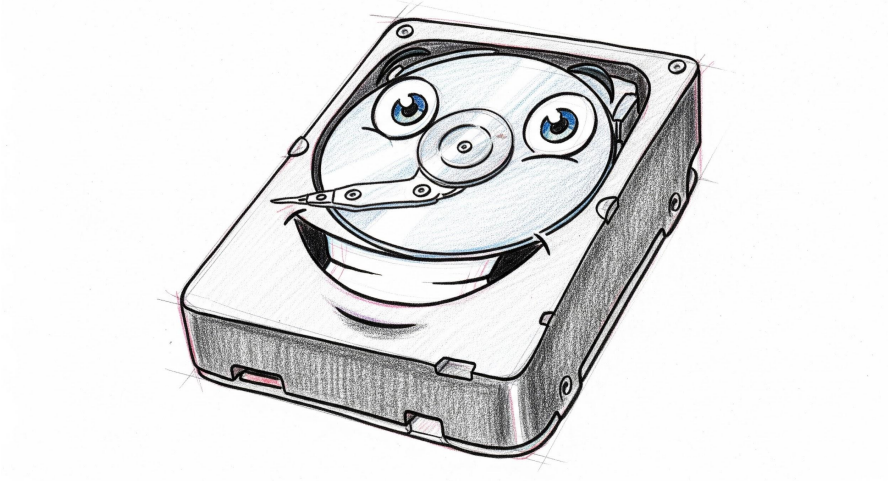
What do we want?

1. Support online volume expansion

2. Support offline volume shrinking and/or modification of the underlying storage

Requirements:

- Native Kubernetes interaction
- Declarative, on-demand, and toil-free
- Test coverage to validate the solution



Why are we doing this?

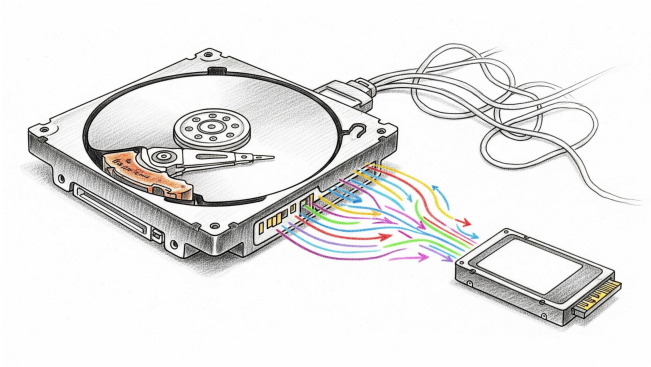
Is this worth doing at all?

To reduce cost

- Scale downs were impossible
- Leading to steady accumulation of overhead
- Eventually the ROI could be justified

To right-size for usage

- Scale ups were manual and risky
- Workloads change over time but our disks stayed the same
- Reduce operational toil



Tackling our first objective: Volume expansion

Getting started

Let’s take back control

- First step is to remove our dependence on volume claim templates in the StatefulSet
- We create a new Custom Resource Definition (CRD) and move our volume claim templates there

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: web
spec:
  selector:
    matchLabels:
      app: nginx
  serviceName: "nginx"
  replicas: 3
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        ...
  volumeClaimTemplates:
  - metadata:
      name: www
    spec:
      accessModes: [ "ReadWriteOnce" ]
      storageClassName: "my-storage-class"
      resources:
        requests:
          storage: 1Gi
```

Getting started

Let’s take back control

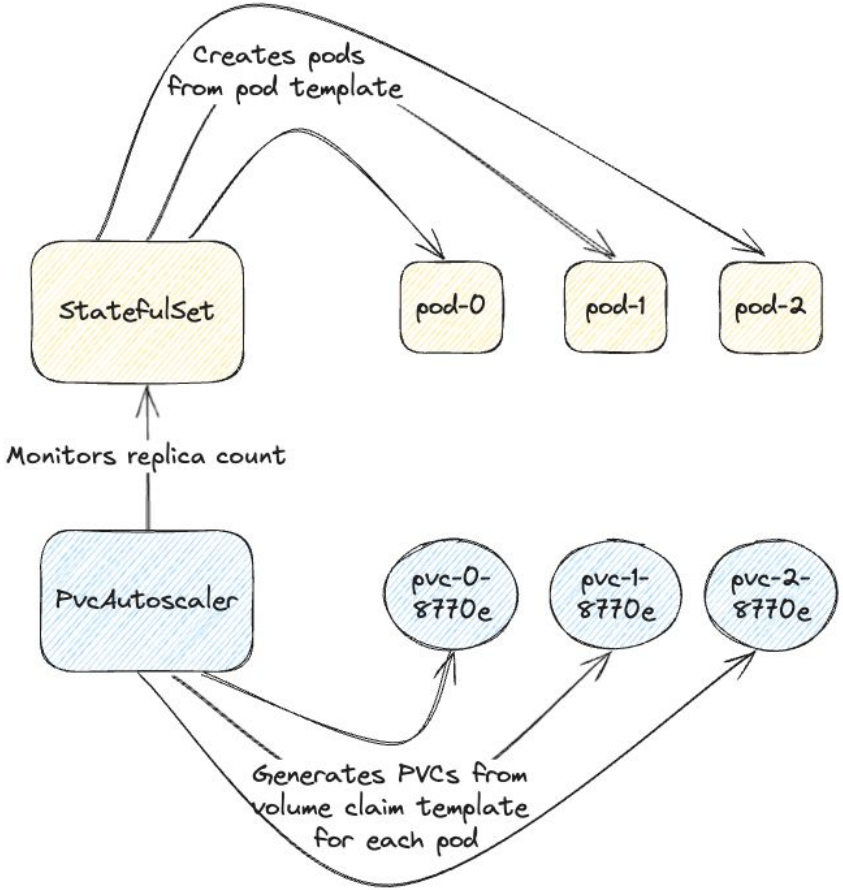
- First step is to remove our dependence on volume claim templates in the StatefulSet
- We create a new Custom Resource Definition (CRD) and move our volume claim templates there
- This allows us to make the volume claim template properties mutable
- By creating a custom Kubernetes operator which owns the PvcAutoscaler resource we can control what happens when you modify the resource

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: web
spec:
  selector:
    matchLabels:
      app: nginx
  serviceName: "nginx"
  replicas: 3
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        ...
---
apiVersion: storage.css.com/v1
kind: PvcAutoscaler
metadata:
  name: web
spec:
  statefulSetName: web
  volumeClaimTemplate:
    metadata:
      name: www
    spec:
      accessModes: [ "ReadWriteOnce" ]
      storageClassName: "my-storage-class"
      resources:
        requests:
          storage: 1Gi
```

The PvcAutoscaler

Unlocking scale ups

- PVCs are now owned by PvcAutoscaler
- Since we own PvcAutoscaler we can allow changes to disk size
- But there's a problem: Pods no longer get volumes for the PVCs generated by the StatefulSet



Admission Webhooks in Kubernetes

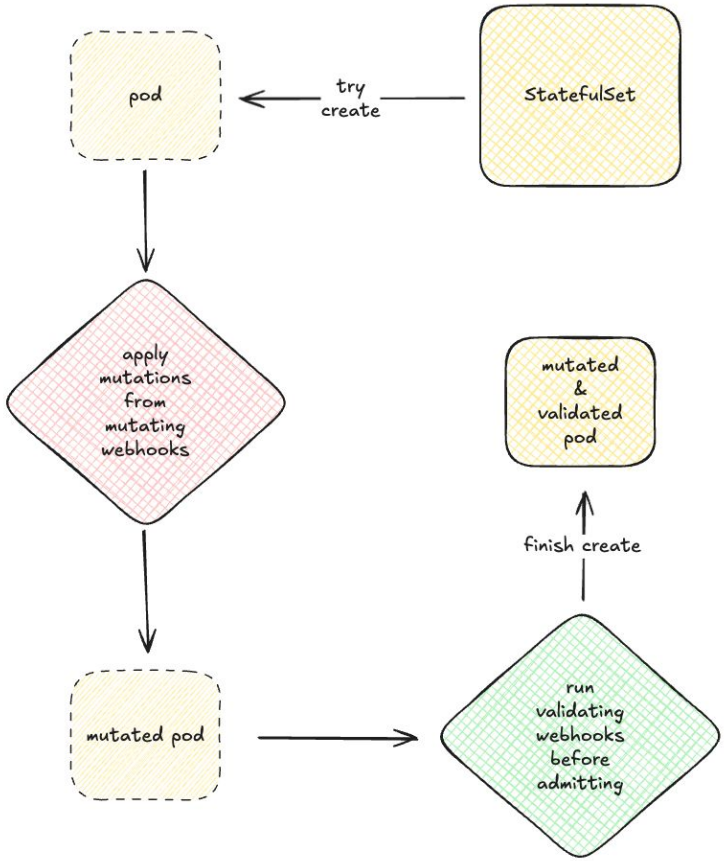
A quick detour

Admission webhooks provide two capabilities in the lifecycle of K8s resources

1. The ability to validate the resource and approve/reject its creation/update/patch/deletion

2. The ability to mutate the resource prior to creation/update/patch

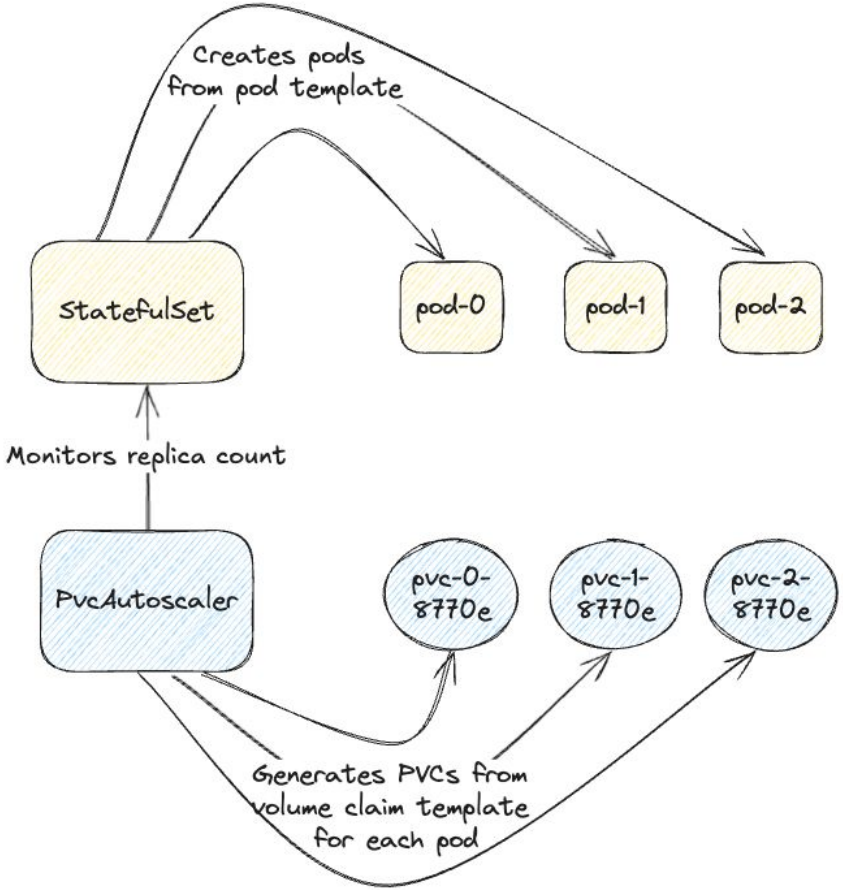
We can use this to inject custom behaviour into the lifecycle of K8s resources



The PvcAutoscaler

Unlocking scale ups

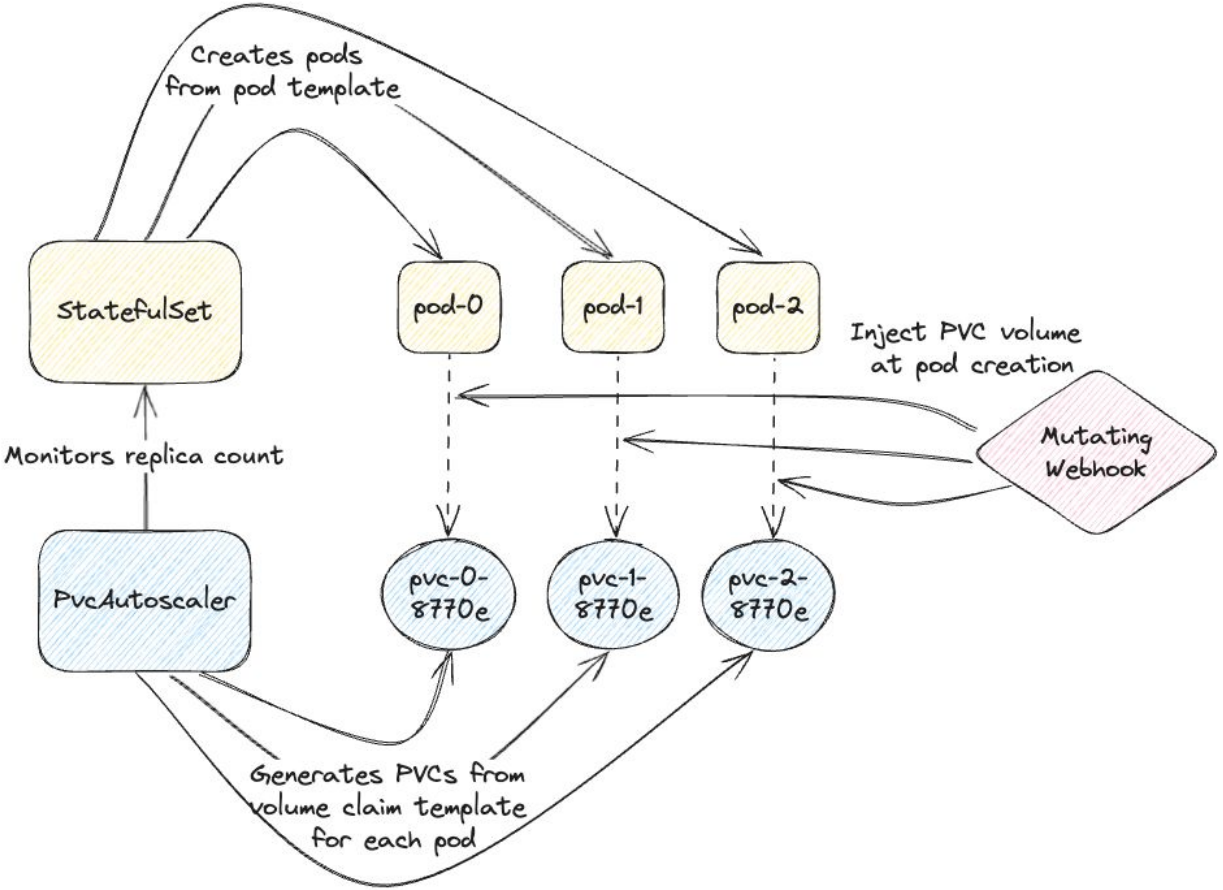
- PVCs are now owned by PvcAutoscaler
- Since we own PvcAutoscaler we can allow changes to disk size
- But there's a problem: Pods no longer get volumes for the PVCs generated by the StatefulSet



The PvcAutoscaler

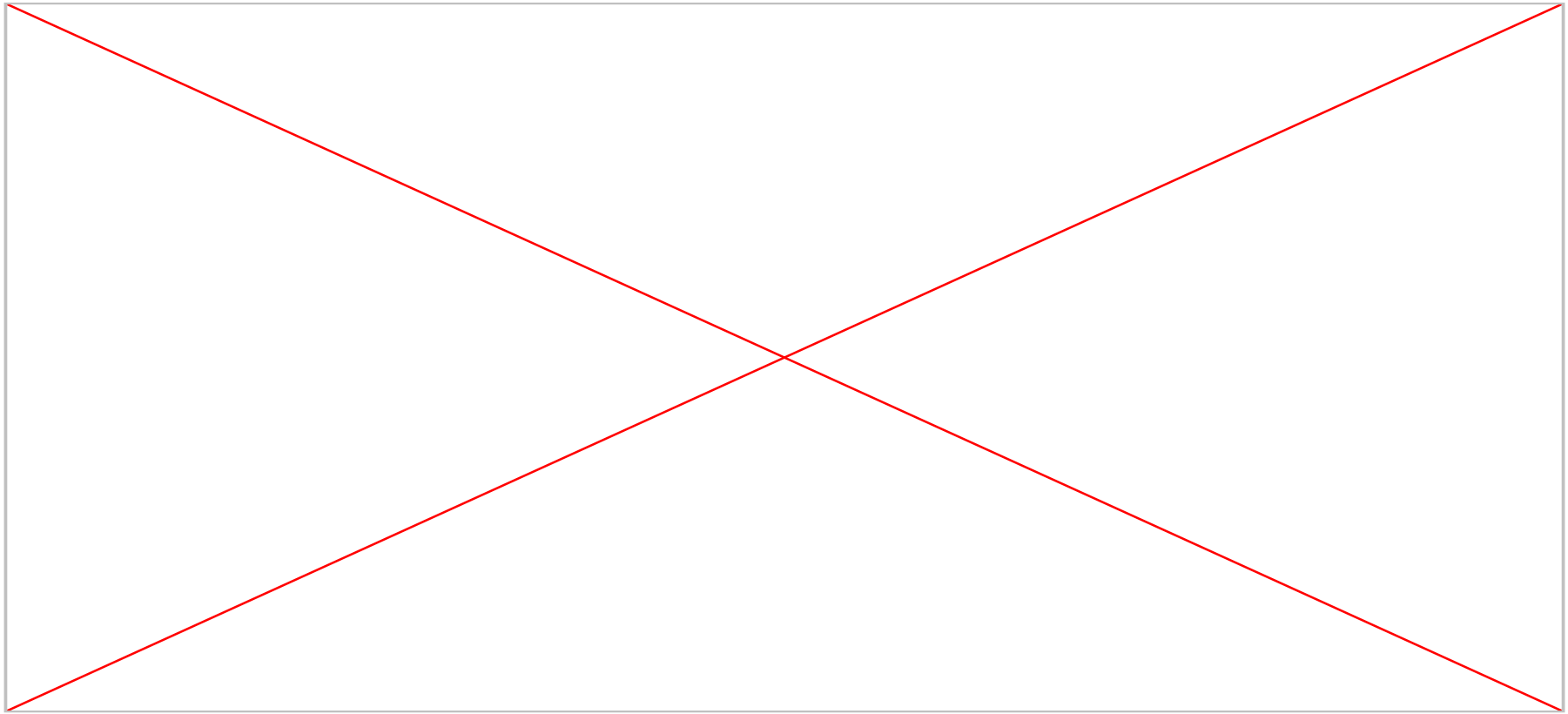
Unlocking scale ups

- PVCs are now owned by PvcAutoscaler
- Since we own PvcAutoscaler we can allow changes to disk size
- Our solution: We'll inject the volumes before the pod is created using a mutating webhook



Demo

Scale up in action



What about second objective?
Volume shrinking & modification

Kubernetes PVCs

Unlocking scale downs

- There is no mechanism to reduce the size or change the underlying storage of a PVC

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc
  namespace: my-namespace
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
```

```
spec.resources.requests.storage: Forbidden: field
can not be less than status.capacity
```

Kubernetes PVCs

Using DataSourceRefs

- DataSourceRefs can be used to enable more custom bootstrapping of PVCs
- CSI driver will do nothing when it sees a dataSourceRef it doesn't know

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: new-pvc
  namespace: my-namespace
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
  dataSourceRef:
    name: volume-populator
    kind: PvcSourcePopulator
    apiGroup: storage.ck.com/v1
```

Kubernetes PVCs

Using DataSourceRefs

- DataSourceRefs can be used to enable more custom bootstrapping of PVCs
- CSI driver will do nothing when it sees a dataSourceRef it doesn't know
- We introduced our own PvcSourcePopulator

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: new-pvc
  namespace: my-namespace
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
  dataSourceRef:
    name: volume-populator
    kind: PvcSourcePopulator
    apiGroup: storage.ck.com/v1
```

Kubernetes PVCs

Using DataSourceRefs

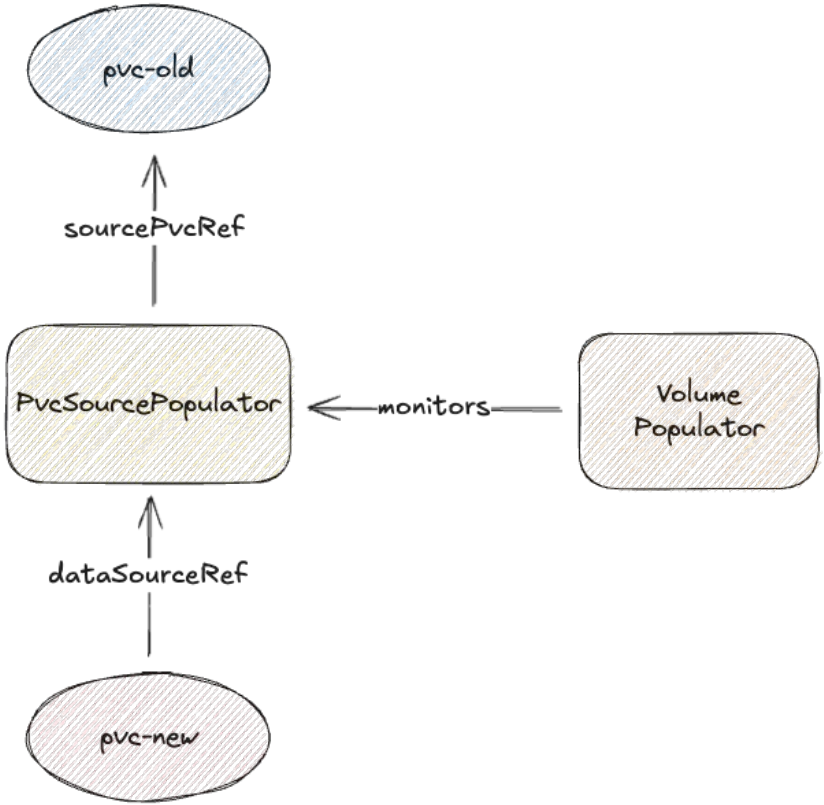
- DataSourceRefs can be used to enable more custom bootstrapping of PVCs
- CSI driver will do nothing when it sees a dataSourceRef it doesn't know
- We introduced our own PvcSourcePopulator
- Enables us to control how the PVC is bootstrapped

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: new-pvc
  namespace: my-namespace
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
  dataSourceRef:
    name: volume-populator
    kind: PvcSourcePopulator
    apiGroup: storage.ck.com/v1
---
apiVersion: storage.css.com/v1
kind: PvcSourcePopulator
metadata:
  name: volume-populator
  Namespace: my-namespace
spec:
  sourcePvcRef:
    name: source-pvc
```

The Volume Populator

Unlocking scale downs

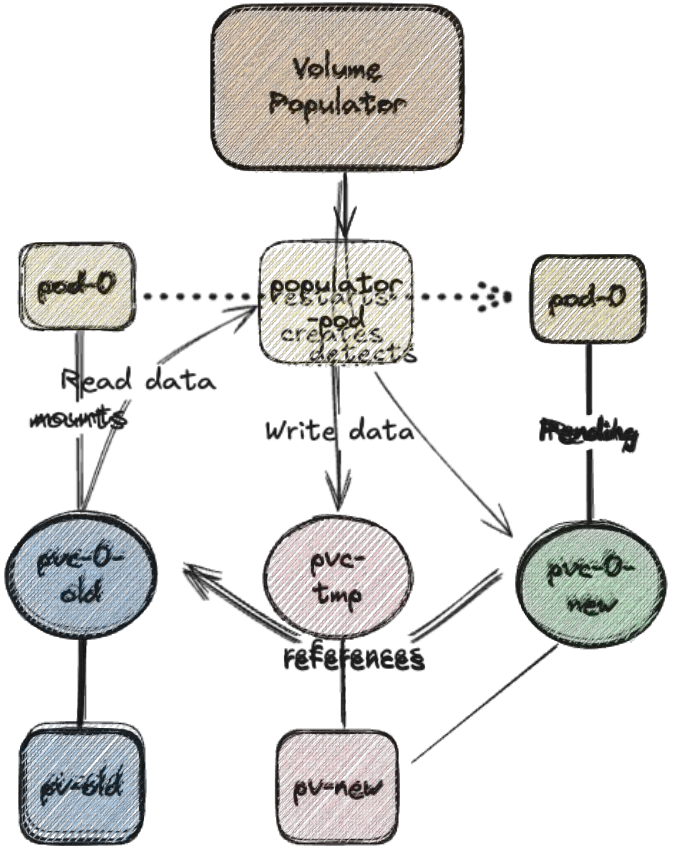
- Now we have a mechanism to reference another PVC when creating a new one
- We can implement a operator that monitors our custom resource (PvcSourcePopulator)
- And implements custom bootstrapping logic
- Now we have all the tools to unlock scale down



The Volume Populator

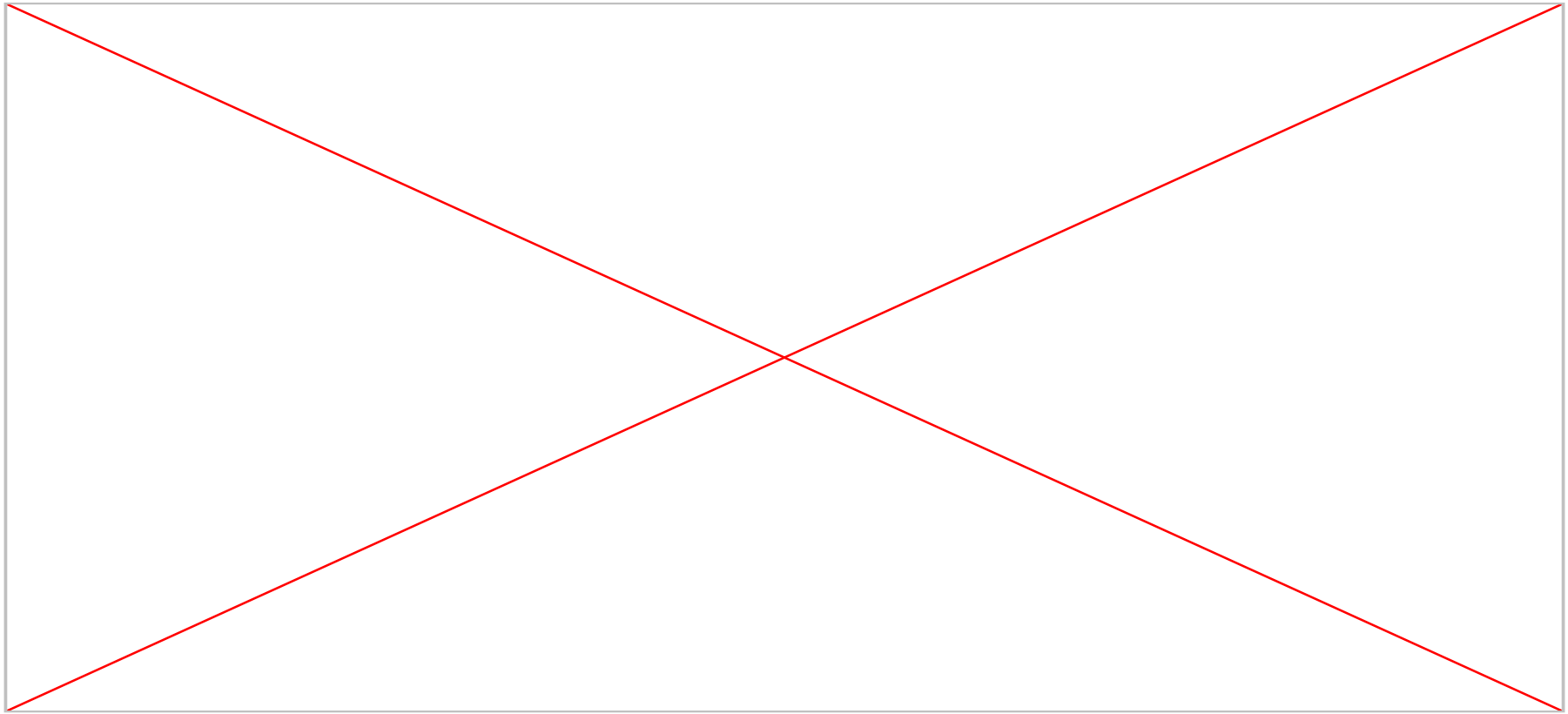
Unlocking scale downs

- Disk size reduced via PvcAutoscaler
- New PVC created with Data Source Ref
- Volume Populator monitors PVCs getting mounted
- New temporary PVC created
- Creates pod used to transfer data between PVCs
- Underlying PV is bound to new PVC



Demo

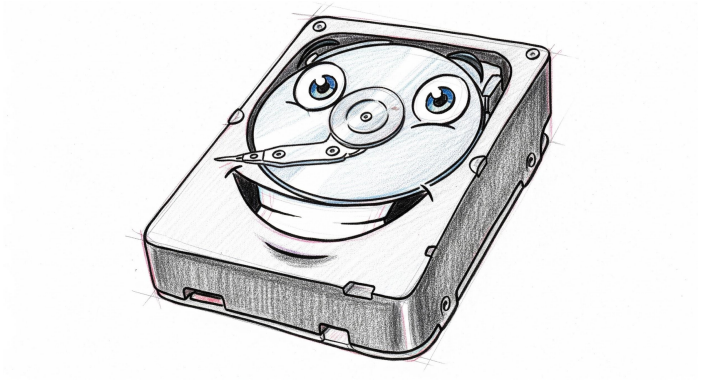
Scale down in action



The Conclusion

What did we achieve?

- Declarative disk management
- Testable disk operation flows
- Risk free disk manipulation - no manual processes
- Lower costs by allowing easy right-sizing
- Caveats: disk shrinking is offline
- Paving the way for automated disk resizing based on metrics

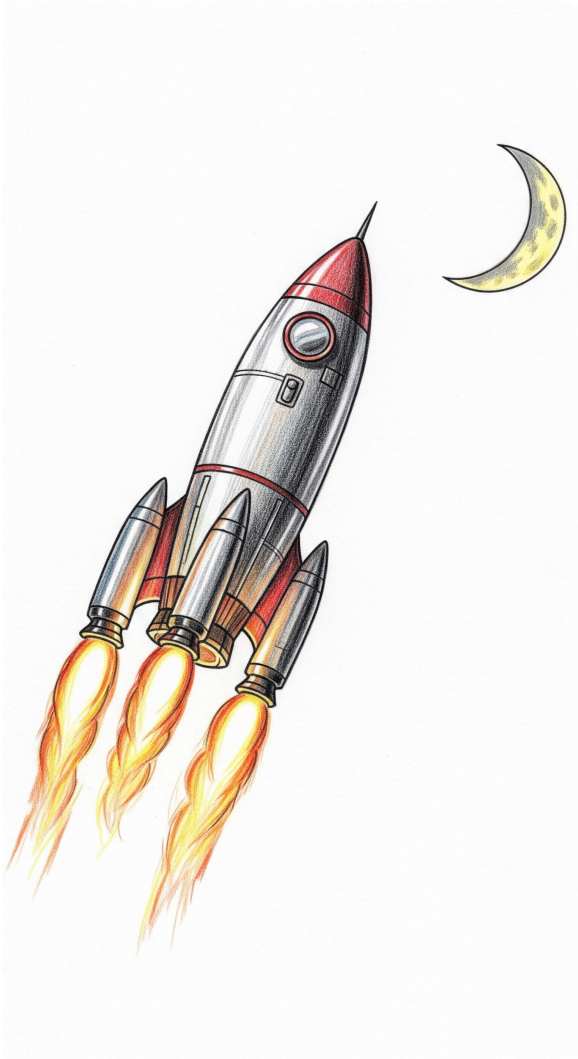


Key Takeaways

What did we learn from doing all of this?

- By understanding the ecosystem we can build entirely new behaviour directly into Kubernetes
- Kubernetes already provides powerful primitives
  - Custom Resource Definitions (CRDs)
  - Admission Control Webhooks
  - Extendable primitives such as the dataSourceRef in PVCs
- The community is continuously pushing to make more resources accessible

Find more about our work on our blog
https://techblog.cloudkitchens.com/



3. Q&A

Thank you!